



Ghost Kitchen

Three AI agents built a restaurant booking app. A table never goes to two parties.

 **tablekeeper**

Book a table. It stays yours.

Book a table

My reservations

✓ 1 Restaurant

✓ 2 Date & party

✓ 3 Time

✓ 4 Name

You're booked.

The table is yours. Nobody else can book it for this time.

Restaurant

Juniper & Rye, table T6

When

Sat, Oct 10, 2026, 5:30 PM PDT

Party of

10

Name

Ava Thompson

Book another table

TABLEKEEPER

#B809A3

TABLE AT

JUNIPER & RYE

DATE

SAT, OCT 10

PARTY

10 GUESTS

TIME

5:30 PM

NAME

AVA THOMPSON

One table, one party
Two people can't
book the same table.

BOOKED

tablekeeper, by Ghost Kitchen. Three AI agents built it, and one of them spent the whole time trying to break it. · [Admin](#)

The assignment

Build a booking site where a table can never be double-booked.

Guests browse restaurants, check a time, book, and cancel. The catch: it has to hold when **two people click at the same second**, when a request is **sent twice**, and when restaurants sit in **different time zones**.

And the app isn't really the entry. The team that built it is.

Three AI agents. One job each.

AGENT	DOES	IS NOT ALLOWED TO
Planner	Splits the assignment into tasks and writes how anyone could tell each one is finished	Write code
Builder	Writes the code and hands each task over with the commands to check it	Mark its own work done
Checker	Tests the work, tries to break it, reports exactly what failed	Fix anything itself

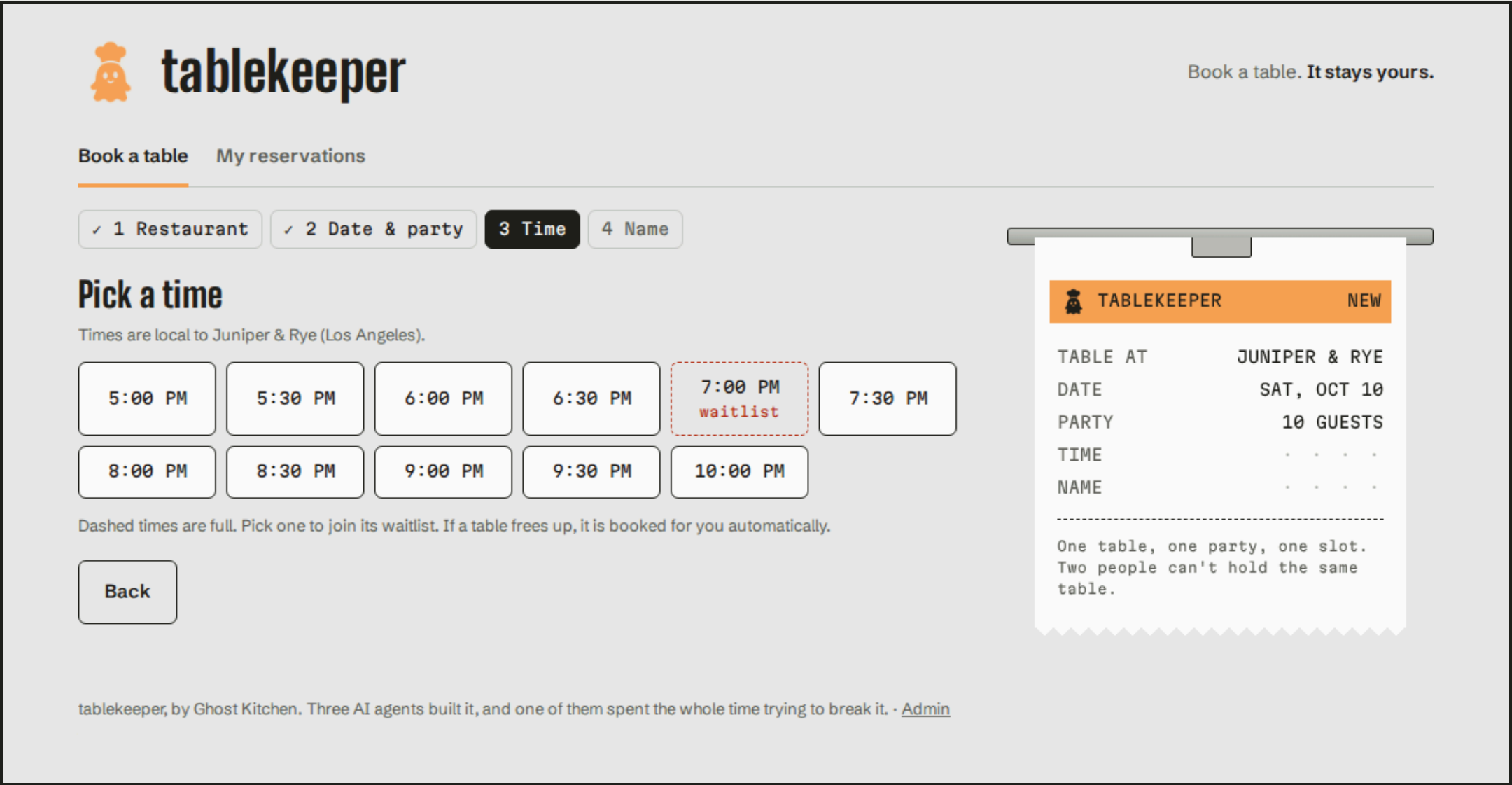
Why split it up

An agent that grades its own work gives itself an A.

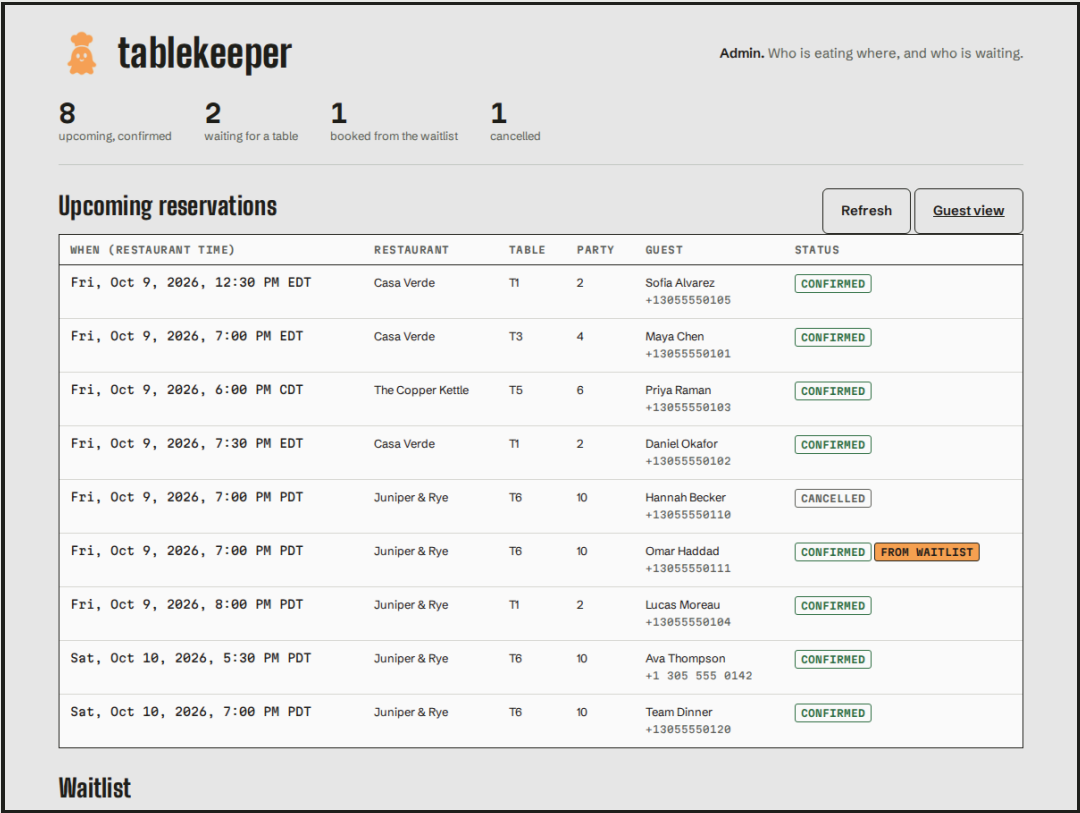
So "done" means **someone else actually saw it work.**

The job descriptions never mention restaurants, so the same three can be pointed at a different assignment. A person hands over the assignment and steers between steps; the agents plan, build and check.

What it shipped: tablekeeper



Pick a time. Full times offer the waitlist. The ticket on the right fills in as you go.



Admin: a cancellation, and the guest moved up from the waitlist onto that table.

The hard part

Four things stop a double booking.

- 1. **One booking at a time.** The database locks while a booking is checked and saved.
- 2. **The database refuses duplicates.** One table, one slot, one confirmed booking.
- 3. **Repeats are recognized.** A request sent twice books once.
- 4. **Only real slots.** 7:00 or 7:30, never 7:15, so nothing overlaps quietly.

RACE TEST		x50
BOOKINGS SENT		50
SAME TABLE		YES
SERVER PROCS		4
GOT THE TABLE		1
TOLD "TAKEN"		49
HELD		

A test that can't fail proves nothing. So we broke the app on purpose.

WHAT WE REMOVED	GOT THE TABLE	
Nothing	1	held
The lock	1	the database rule still holds
The database rule	1	the lock still holds
Both	4	in our run: double-booked, and the test caught it

Did review change anything? Yes.

The Checker rejected the work three times.

Run 1's Checker approved broken work. An audit caught it, and its instructions were rewritten: prove every safety test can fail. In run 2, an independent Checker found 7 problems. All were fixed before it passed. In run 3, the Checker found 2 more problems. All were fixed before it passed. In run 4, the Checker found no problems. The work passed.

ROUND 1	ROUND 2	ROUND 3	ROUND 4
ODD DATES CRASH MALFORMED REQUESTS CRASH LONG NAME BREAKS PHONE LAYOUT 2 TESTS CAN'T FAIL	FIXES 1 TO 4 OK SHOWS TIMES IT WON'T BOOK	SAME, IN THE UI TESTS BREAK A FEW DAYS A YEAR	ALL 7 FIXED TEST DATES CHECKED HOURLY FOR 3 YEARS
FAIL ×4	FAIL ×1	FAIL ×2	PASS

What you can check yourself

Everything is in the repo.

64

checks pass, plus the break-it-on-purpose test

3

job descriptions, none mentioning restaurants

0

network needed: every check runs with networking switched off

1

transcript with every plan, handoff and verdict, including the wrong ones



Book a table. It stays yours.

github.com/yasmeen/hackathon-dark-factory